

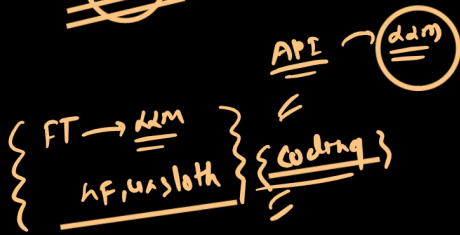
## Decoder

- ① Positional encoding
- ② Self Attention  $\longrightarrow$  Multihhead Self Attention
- ③ layer normalization

$\approx$  { Cross Attention  
 $\succ$  { Self mask attention

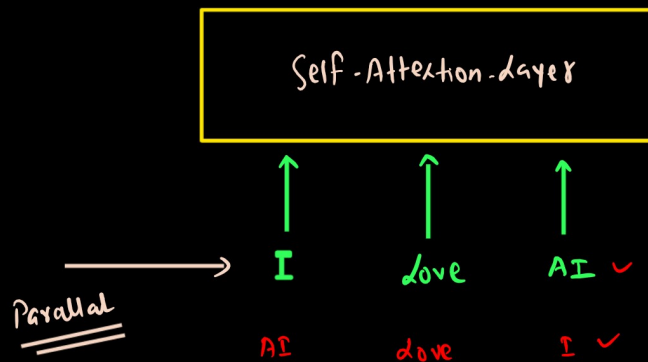
end2end flow encoder  $\longrightarrow$  decoder

2-3



## Positional encoding

Problem statement → Data → Parallelly Solve → Sequence → PE



### Positional Encoding (Definition)

Positional encoding is a technique used to inject information about the position (order) of tokens into their embeddings, so that the model can understand sequence order.

Why it is needed?

Transformers process all tokens in parallel

So they don't know order by default

Example:

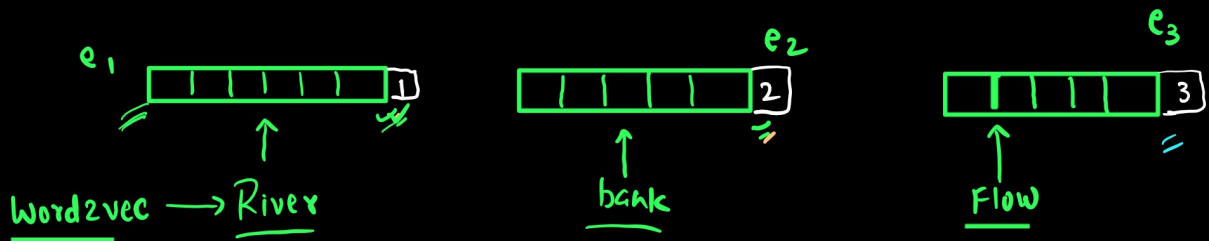
- "I love AI"
- "AI love I"

Without positional encoding → both look same to model

Dog bites man

man bites dog

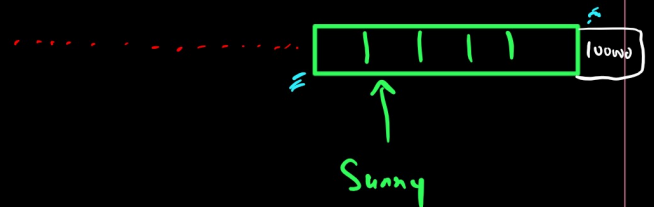
# Self-Attention-Layer



Real time data

Self  $\rightarrow$  100000

No-limit  $\leq$



Simple-Approach

Self  $\rightarrow$  4 token

I Love AI Much  
1 2 3 4

Self  $\rightarrow$  4w token

I Love AI Much again ..... You  
1 2 3 4 5 ..... 4w

Self  $\rightarrow$  Limitless

n token

1L, 10L, ...

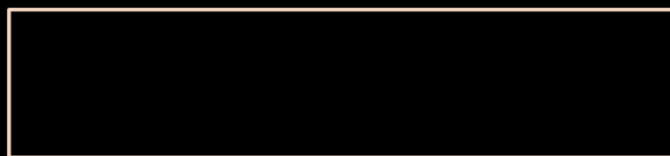
Drawback

① No of token could be limitless

② Discrete (1, 2, 3, 4, 5, ..., 1000, 10000)  $\rightarrow$  vector

NN

(training might become unstable)



← this vector having a knowledge of pos

Positional encoding = PE1

PE vector

Addition

$e_1$

great

Word 2 vec

PE2

$e_2$

indig

Same Dim of vector

Concatination

(APPEND)

Dim ↑↑↑↑↑



10-Dim

10-Dim

NN / Attention

more computational Power

ADDITION

$e_1 + PE1$

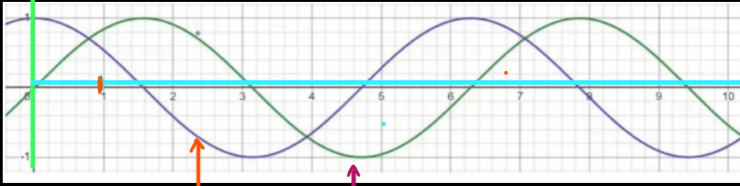


Same dim of vector



PE1

Encoded value



cos  
[-1 to +1]

sin → [-1 to +1]

PE2

9-10th class

School

Pythagoras

TRIGONOMETRY  
Sin, cos, tan

Position  
of the word



Sin | cos

original RP  
512

))))  
= word2vec

9vec  
1

radia  
2

~~Descarte~~

Continue

E1



sin(Pos)  
→ sin(1) → 0.84  
cos(1) → 0.54  
cos(Pos)  
≈ [0.84 | 0.54]  
2

E2



sin(Pos)  
→ sin(2) → 0.9  
cos(2) → -0.4  
cos(Pos)  
≈ [0.9 | -0.4]  
2

PE2

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}}) \quad \checkmark \quad \sin(Pos) \longrightarrow \sin(Pos | 10000^{2i/d_{model}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}}) \quad \checkmark \quad \cos(Pos) \longrightarrow \cos(Pos | 10000^{2i/d_{model}})$$

Pos  $\longrightarrow$  Position

$d_{model}$   $\longrightarrow$  Dim of embedding model

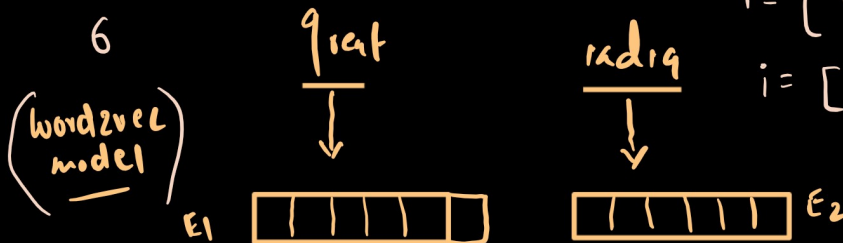
$i$   $\longrightarrow$  Range

$$i = [\text{Low Range} - \text{Upper Range}]$$

$$= [0 - (\frac{d_{model}}{2} - 1)]$$

$$i = [0 - (\frac{6}{2} - 1)]$$

$$i = [0 - 2] \longrightarrow \checkmark 0, 1, 2 \}$$



4 tokens

I Love my country  
Pos  $\longrightarrow$  0 1 2 3

Dim → 6

Pos = 0  
9 real

E<sub>1</sub>



E<sub>2</sub>



Pos = 1  
india

wordvec

d<sub>model</sub> → 6 =

$$i = \left[ 0 - (6/2 - 1) \right]$$

$$= [0 - 2]$$

$$i = 0, 1, 2$$

sin/cos cos/sin sin/cos → ⑥

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

PE<sub>2</sub>

For i = 0 → Pos 0

$$PE(0, 0) = \sin(0/10000^0) = 0$$

$$PE(0, 1) = \cos(0/10000^0) = 1$$

For i = 0 → Pos → 1

$$PE(1, 0) = \sin(1/10000^0) = 0.84$$

$$PE(1, 1) = \cos(1/10000^0) = 0.54$$

For i = 1 2x1

$$PE(0, 2) = \sin(0/10000^{1/3}) = 0$$

$$PE(0, 3) = \cos(0/10000^{1/3}) = 1$$

For i = 1

$$PE(1, 2) = \sin(1/10000^{1/3}) = 0.04$$

$$PE(1, 3) = \cos(1/10000^{1/3}) = 0.99$$

For i = 2

$$PE(0, 4) = \sin(0/10000^{2/3}) = 0$$

$$PE(0, 5) = \cos(0/10000^{2/3}) = 1$$

For i = 2

$$PE(1, 4) = \sin(1/10000^{2/3}) = 0.01$$

$$PE(1, 5) = \cos(1/10000^{2/3}) = 0.98$$

PE<sub>1</sub> [0 | 1 | 0 | 1 | 0 | 1]

PE<sub>2</sub> → [0.84 | 0.54 | 0.04 | 0.99 | 0.01 | 0.98]

Dim 6

E<sub>1</sub>



x<sub>1</sub>

E<sub>2</sub>



→ TODAY 1 AM TEACHING PE

tokens → 5

Pos → 0

1

2

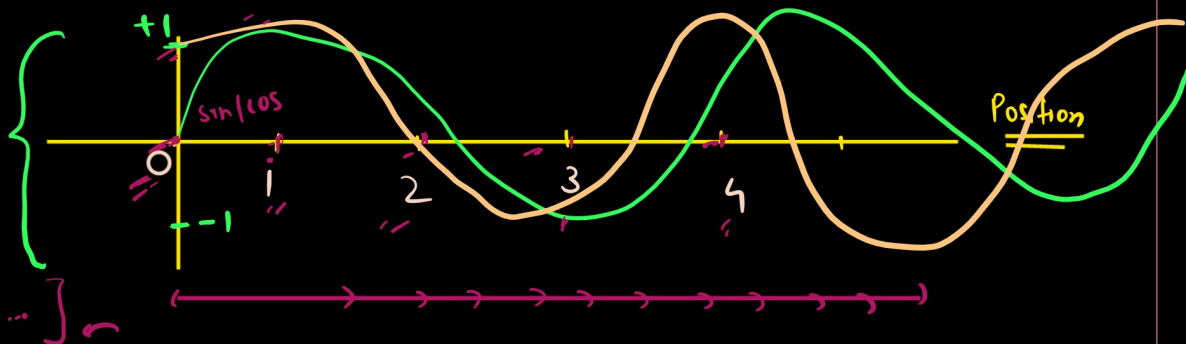
3

4

~~discrete~~  
continuous (sin/cos)

encoded value

seq info



## ② Self Attention → Multihed Attention

Money bank grow

Similarity → Dot Product

$$\text{bank} = w_1 * \text{Money} + w_2 * \text{bank} + w_3 * \text{grow}$$

(q,k)

$$[e_{\text{bank}} \cdot e_{\text{money}}] \text{Money} + [e_{\text{bank}} \cdot e_{\text{bank}}] * \text{bank} + [e_{\text{bank}} \cdot e_{\text{grow}}] * \text{grow}$$

$\underbrace{\quad}_{q} \quad \underbrace{\quad}_{k}$

$$\text{bank} = \left( \underbrace{\begin{pmatrix} \text{Matrix} & \text{Vector} \end{pmatrix}}_{e_{\text{bank}}} \cdot \underbrace{\text{Vector}}_{e_{\text{money}}} \right) * \text{Money} + \left( \underbrace{\begin{pmatrix} \text{Matrix} & \text{Vector} \end{pmatrix}}_{e_{\text{bank}}} \cdot \underbrace{\text{Vector}}_{e_{\text{bank}}} \right) * \text{bank} + \left( \underbrace{\begin{pmatrix} \text{Matrix} & \text{Vector} \end{pmatrix}}_{e_{\text{bank}}} \cdot \underbrace{\text{Vector}}_{e_{\text{grow}}} \right) * \text{grow}$$

$\underbrace{\quad}_{S_1} \quad \underbrace{\quad}_{S_2} \quad \underbrace{\quad}_{S_3}$

SOFTMAX

$$w_1 = \frac{e^{S_1}}{e^{S_1} + e^{S_2} + e^{S_3}}$$

$$w_2 = \frac{e^{S_2}}{e^{S_1} + e^{S_2} + e^{S_3}}$$

$$w_3 = \frac{e^{S_3}}{e^{S_1} + e^{S_2} + e^{S_3}}$$

value vector (v)

$$w_1 * \text{Vector} +$$

$e_{\text{money}}$

(v)

$$w_2 * \text{Vector} +$$

$e_{\text{bank}}$

(v)

$$w_3 * \text{Vector}$$

$e_{\text{grow}}$

{Self → (Q, K, V)}

bank

Contextual vector

$$\begin{bmatrix} 1 & 2 \end{bmatrix} \quad \begin{bmatrix} 3 & 4 \end{bmatrix}$$

$$\begin{matrix} \times \\ \begin{bmatrix} 1 & 2 \end{bmatrix} \\ a_1 \quad b_1 \end{matrix} \quad \begin{matrix} \times \\ \begin{bmatrix} 3 \\ 4 \end{bmatrix} \\ a_2 \quad b_2 \end{matrix}$$

$$(1 \times 3) + (2 \times 4) = 11 \quad \text{Scalar value}$$

$a_1 \times a_2 \quad b_1 \times b_2$

Self-Attention  $\rightarrow$  Q, K, V

$$D = \{a:1, b:2, c:3\}$$

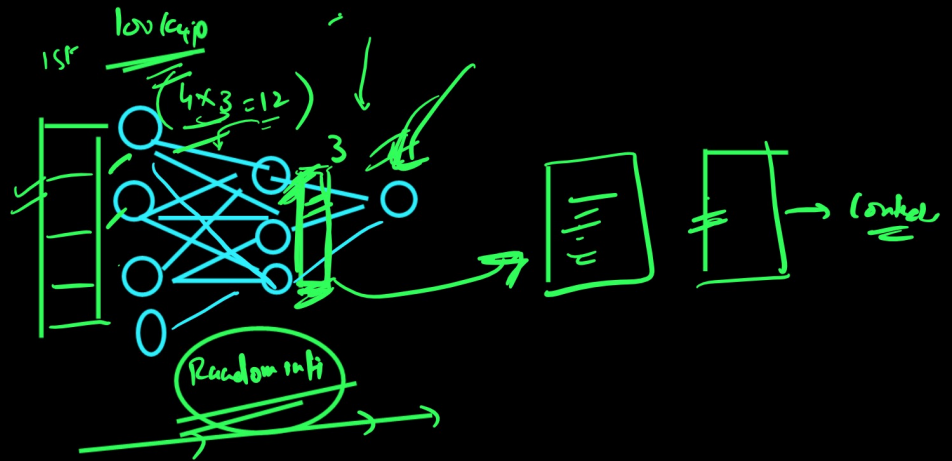
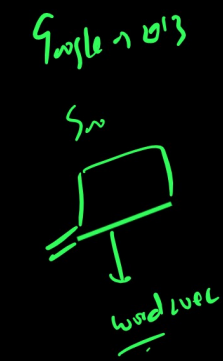
Analogy

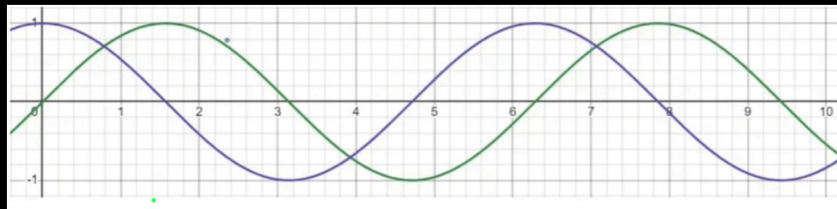
$$D[a] \rightarrow \text{OIP} = 1$$

query      key      value

Self-Attention  $\rightarrow$  extend

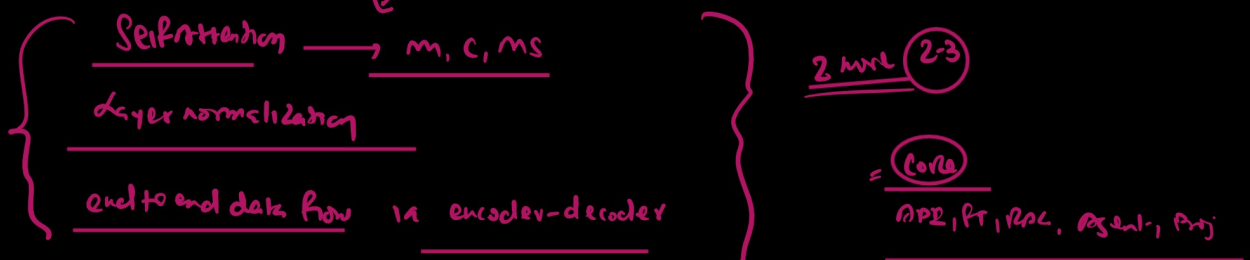
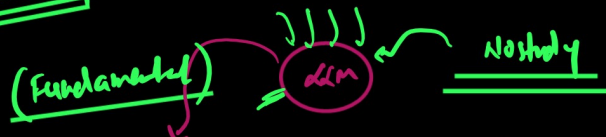
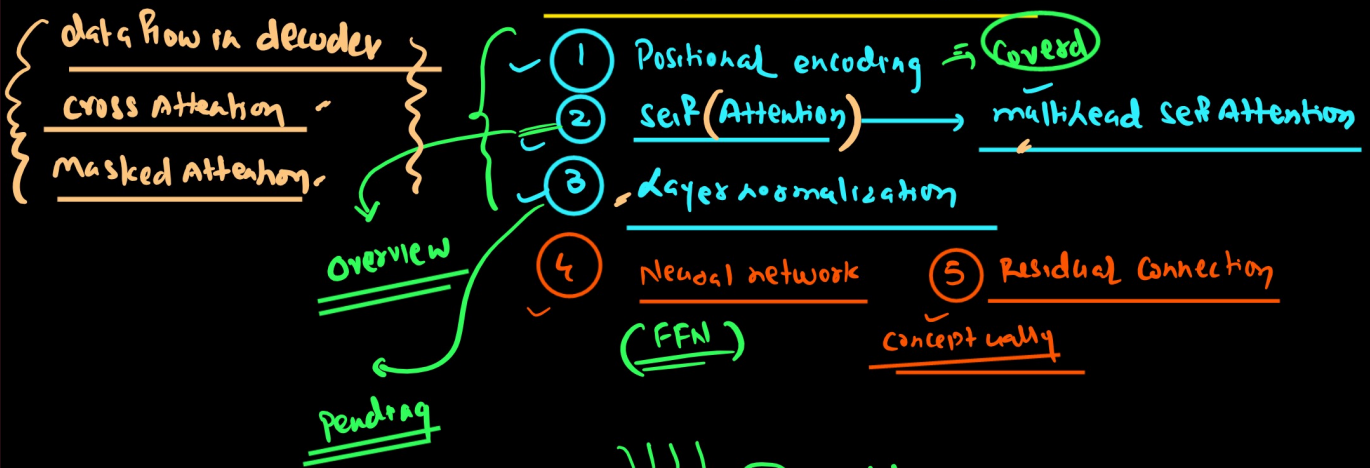
- in self-attention
- Cross Attention
- masked





- ① Encoder
- ② Decoder

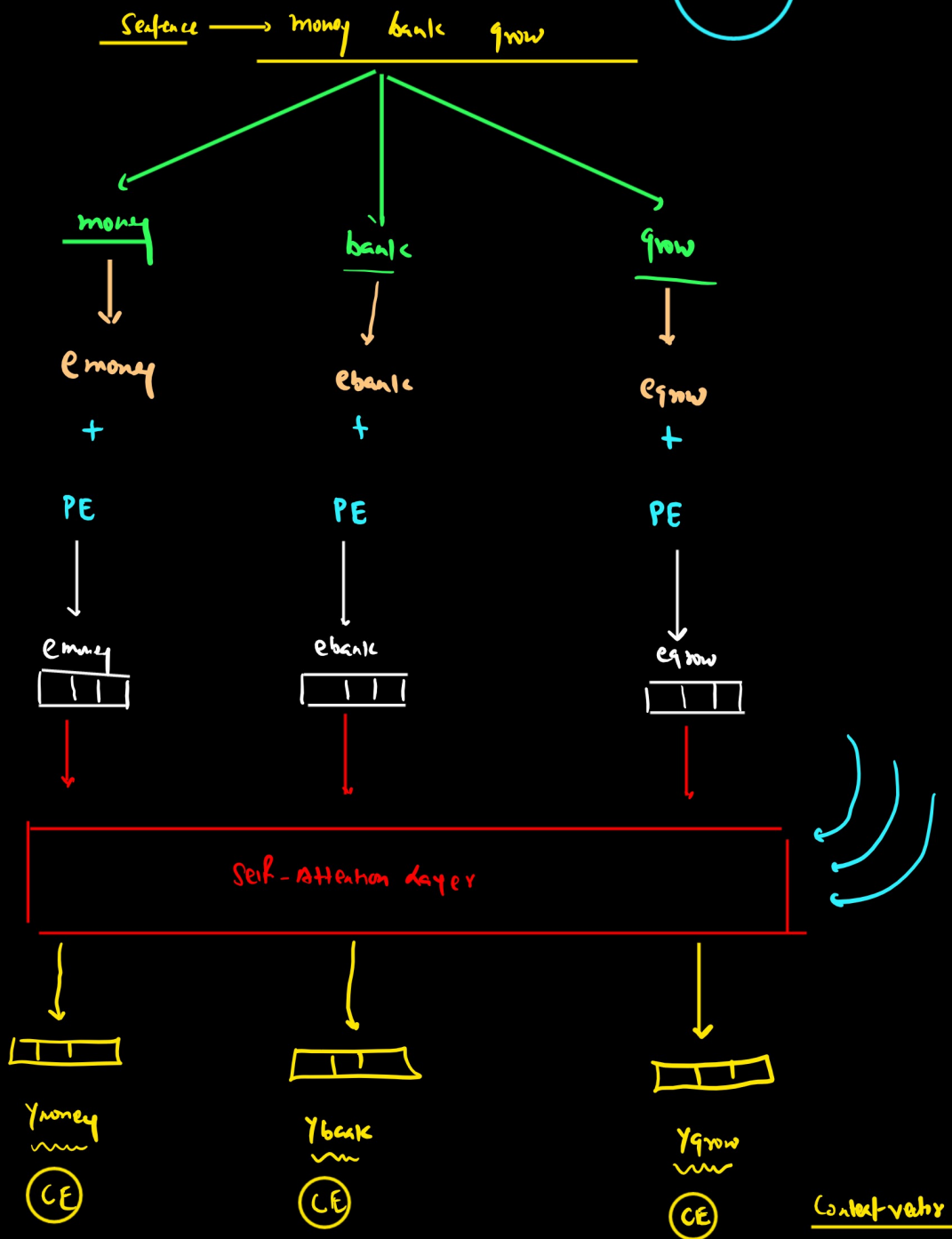
How the data flows in encoder





# Self Attention

Parallel



Similarity score

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Self-Attention → Contextual vector



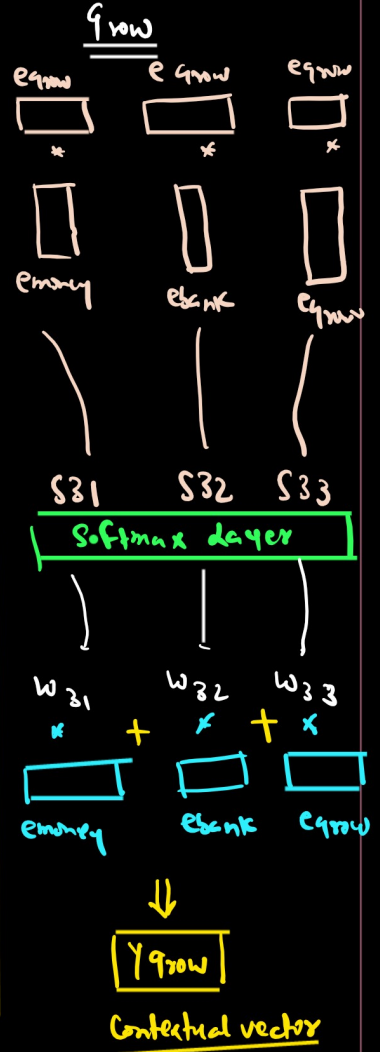
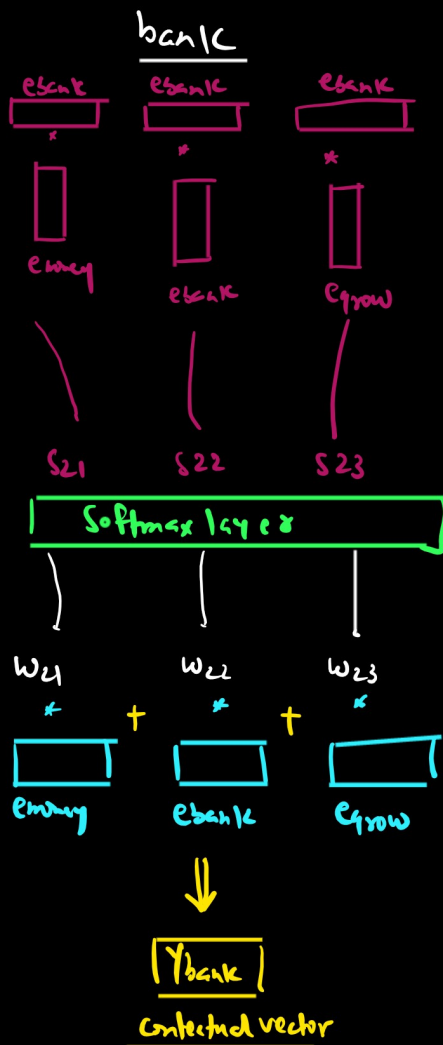
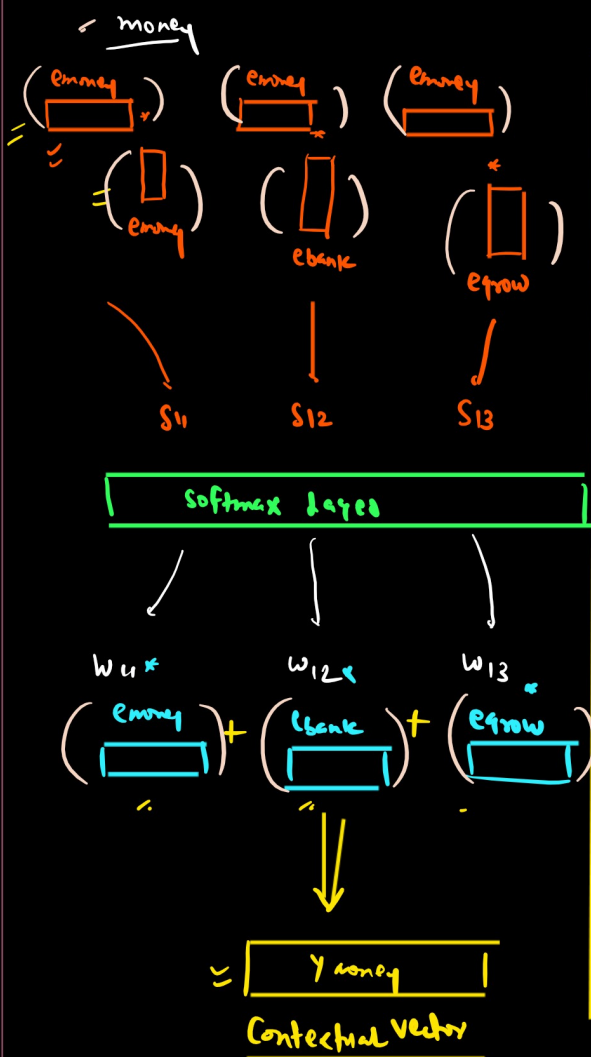
money bank grow  $\longrightarrow$  How this text is being processed using Self Attention

Find out the Relation of each word with word

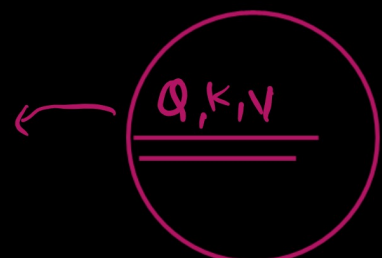
$$\begin{aligned}
 \text{money CE} &= \overbrace{w_{11}}^{\text{similarity score}} * e_{\text{money}} + w_{12} * e_{\text{bank}} + w_{13} * e_{\text{grow}} \\
 \text{bank CE} &= w_{21} * e_{\text{money}} + w_{22} * e_{\text{bank}} + w_{23} * e_{\text{grow}} \\
 \text{grow CE} &= w_{31} * e_{\text{money}} + w_{32} * e_{\text{bank}} + w_{33} * e_{\text{grow}}
 \end{aligned}$$

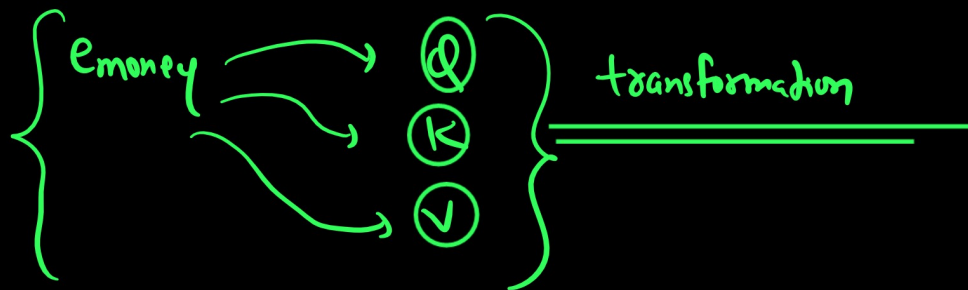
Dot Product  
 $\uparrow$

(SS)



Same vector  $\longrightarrow$  (multiple times)  
(this is confusing)

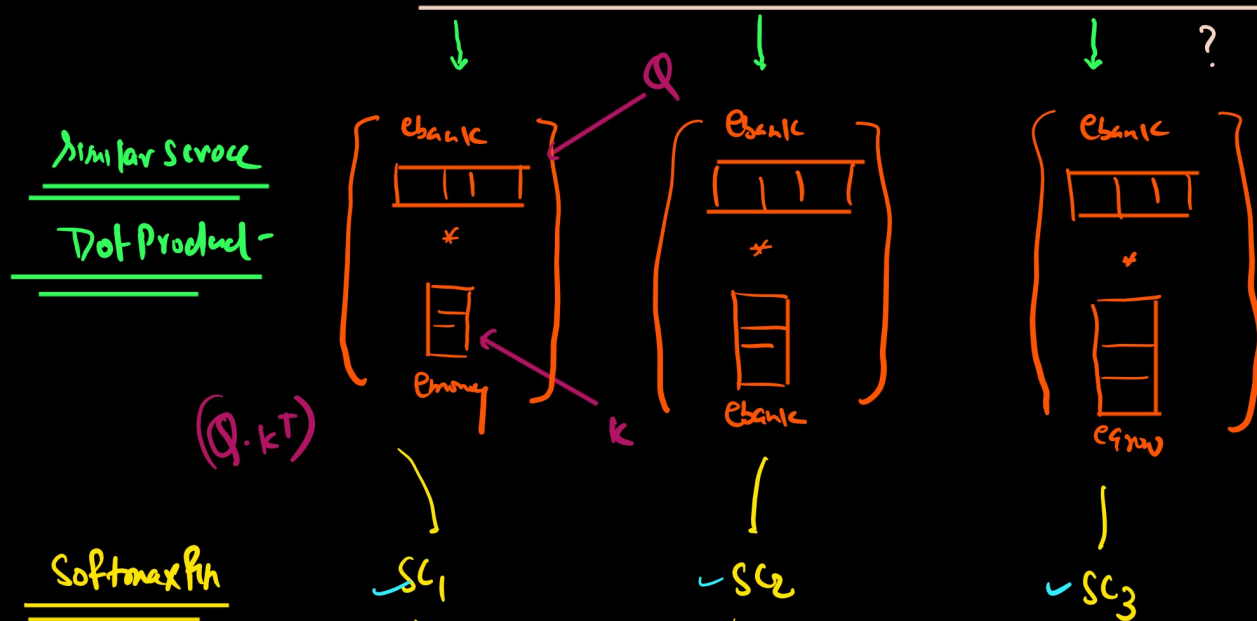




money | bank grow  $\xrightarrow{\text{CE}}$  self-Attention?

map this word in context of other word  $\Leftarrow$

$$\text{bank}_{CE} = w_{11} * \text{emoney} + w_{12} * \text{ebank} + w_{13} * \text{egrow}$$



Softmax layer

$$w_1 = \frac{e^{SC_1}}{e^{SC_1} + e^{SC_2} + e^{SC_3}}$$

$$w_2 = \frac{e^{SC_2}}{e^{SC_1} + e^{SC_2} + e^{SC_3}}$$

$$w_3 = \frac{e^{SC_3}}{e^{SC_1} + e^{SC_2} + e^{SC_3}}$$

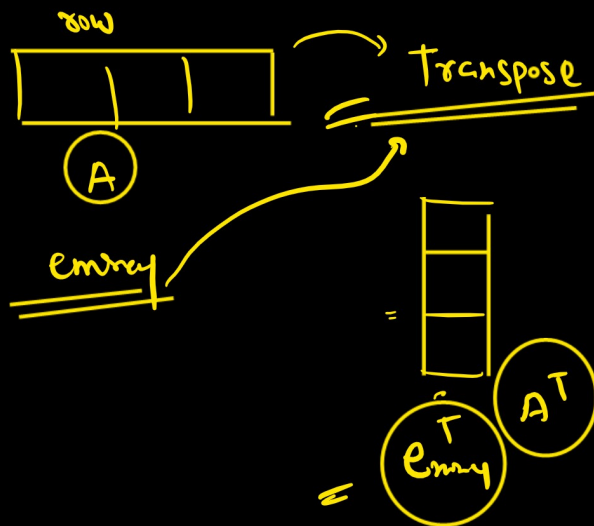
$$w_1 * \begin{bmatrix} | & | & | \end{bmatrix} + w_2 * \begin{bmatrix} | & | & | \end{bmatrix} + w_3 * \begin{bmatrix} | & | & | \end{bmatrix}$$

emoney ebank egrow

$\text{Softmax} \left( \frac{Q \cdot K^T}{\sqrt{d_k}} \right) * \underline{\underline{V_{key}}}$

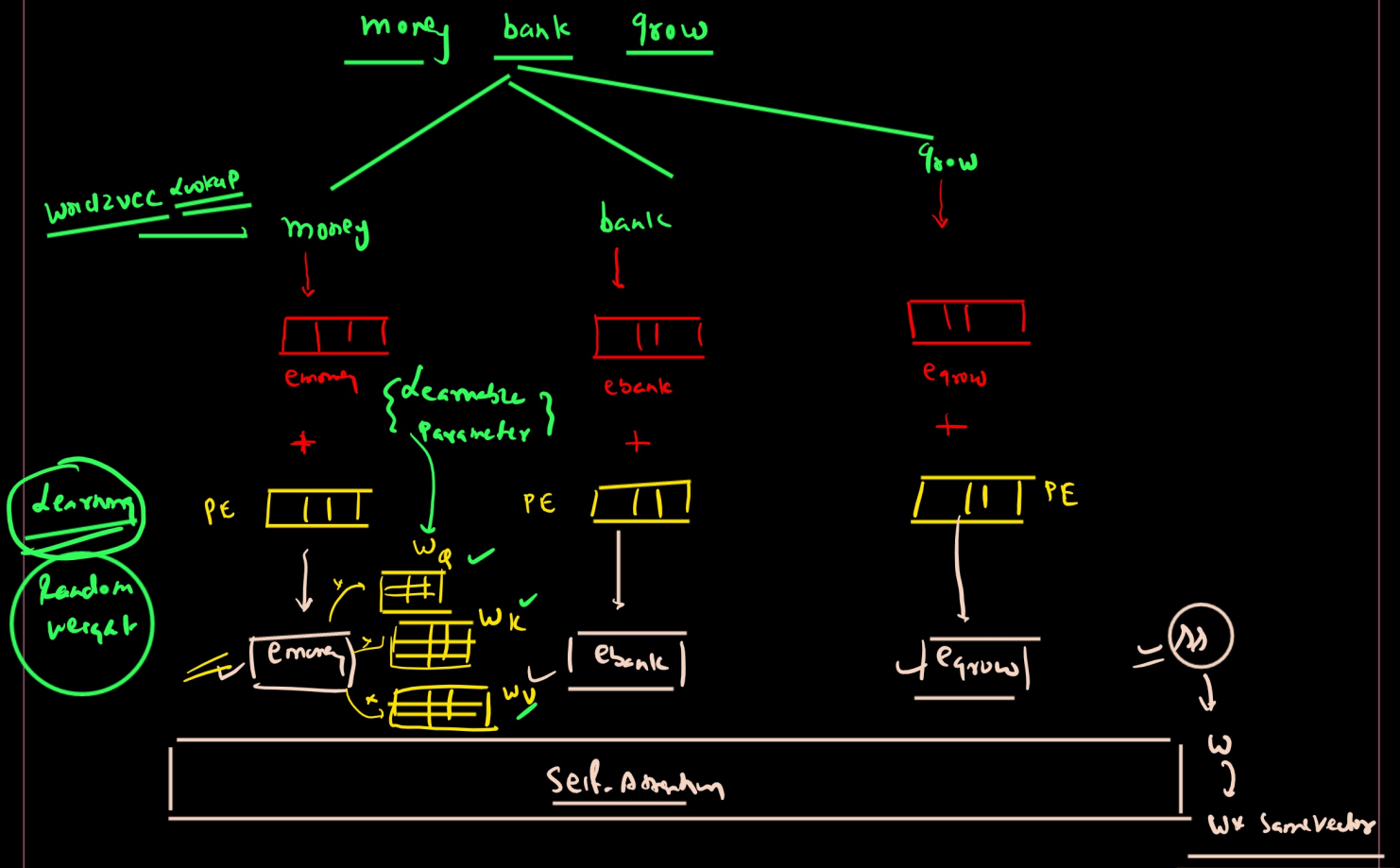
$\begin{bmatrix} | & | & | \end{bmatrix}$  ← Contextual vector

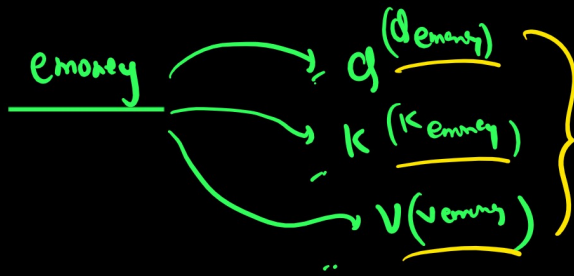
ebank



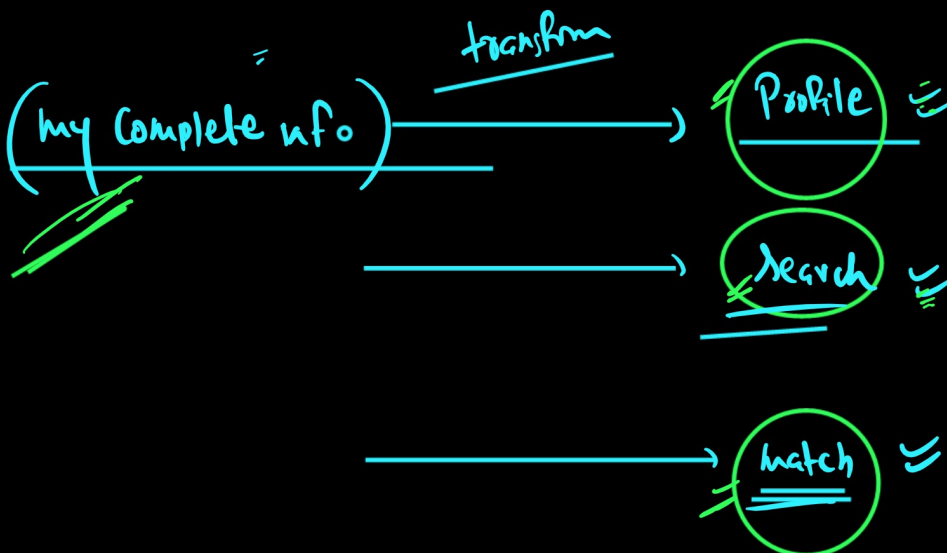
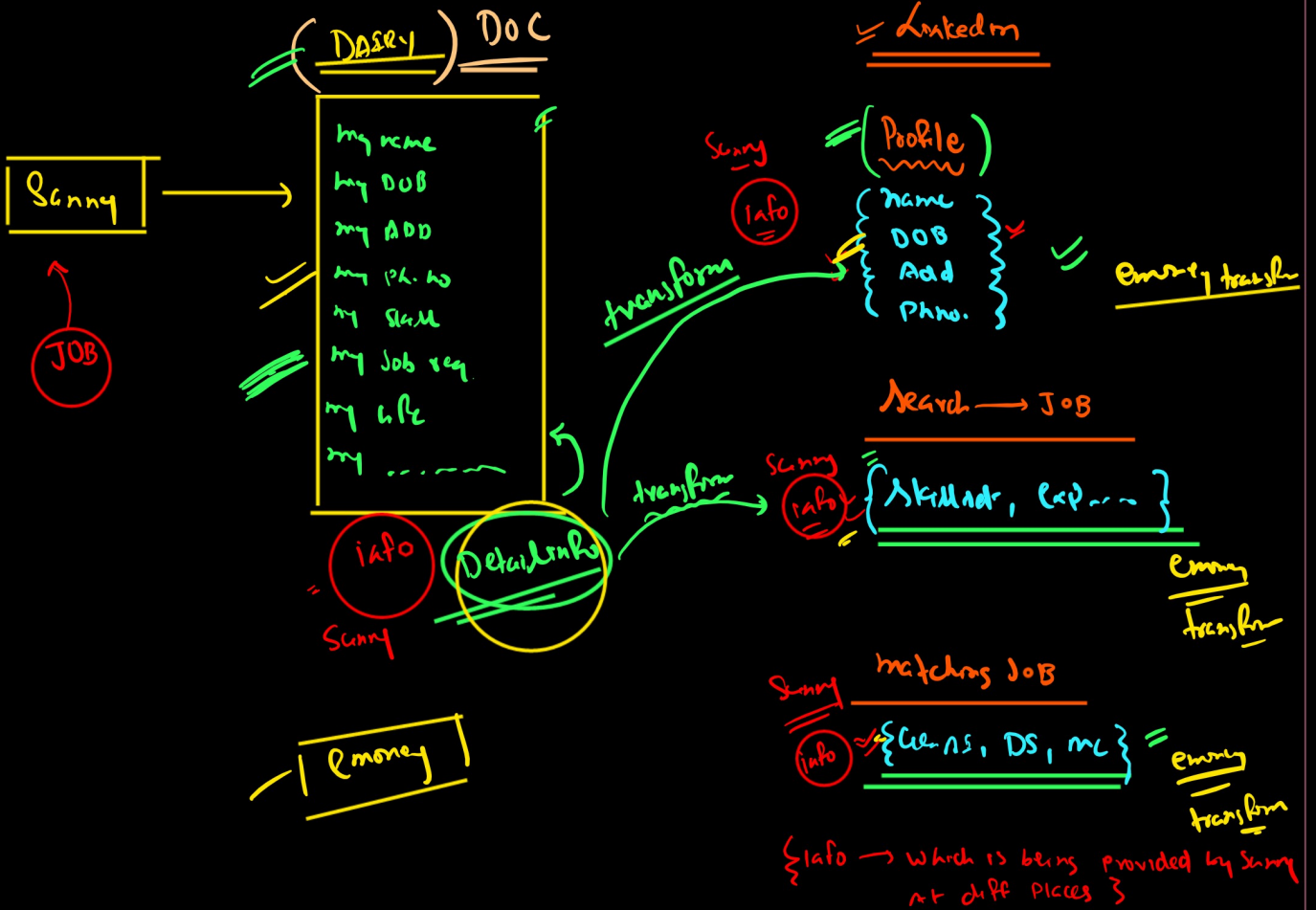
Dot Product

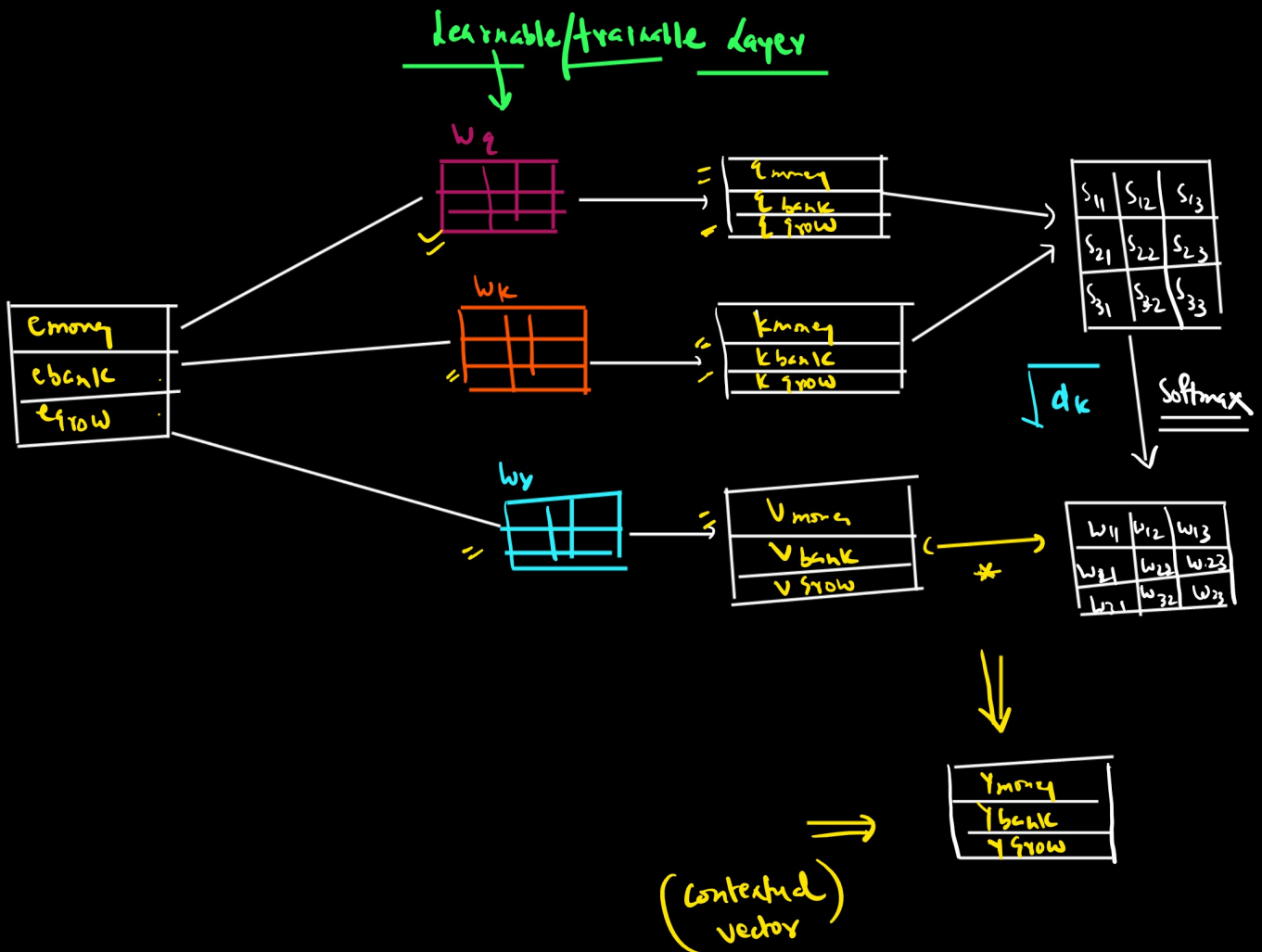
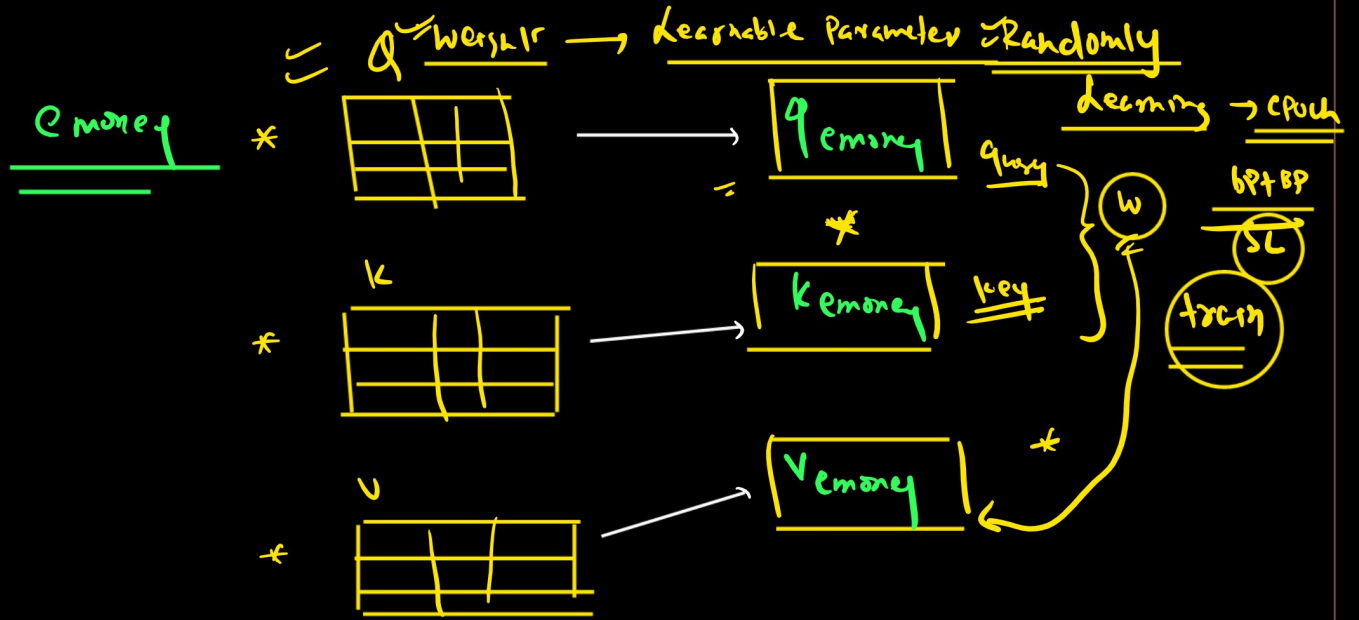
horizontal \* vertical



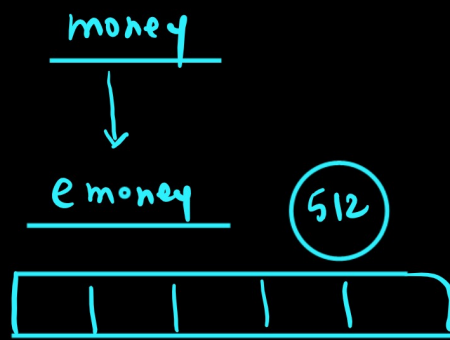


## Analogy

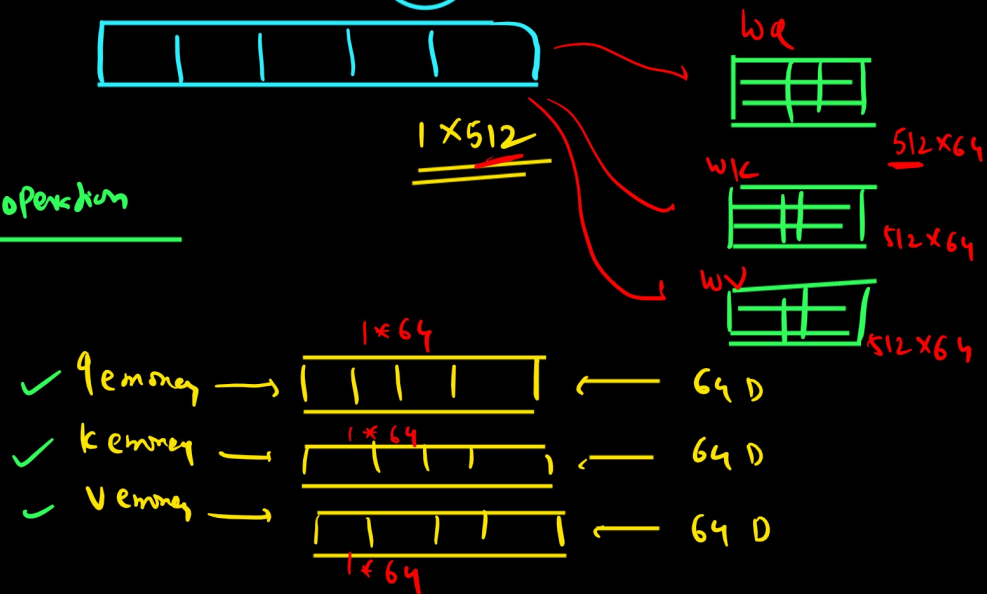








basic matrix operation



(64)

qemoney \* kemoney = value ?

↓  
✗

Softmax

$\left( \frac{\text{value}}{\sqrt{d_k}} \right) = (\text{value}')$

↓  $\sqrt{64} = 8$

dimension

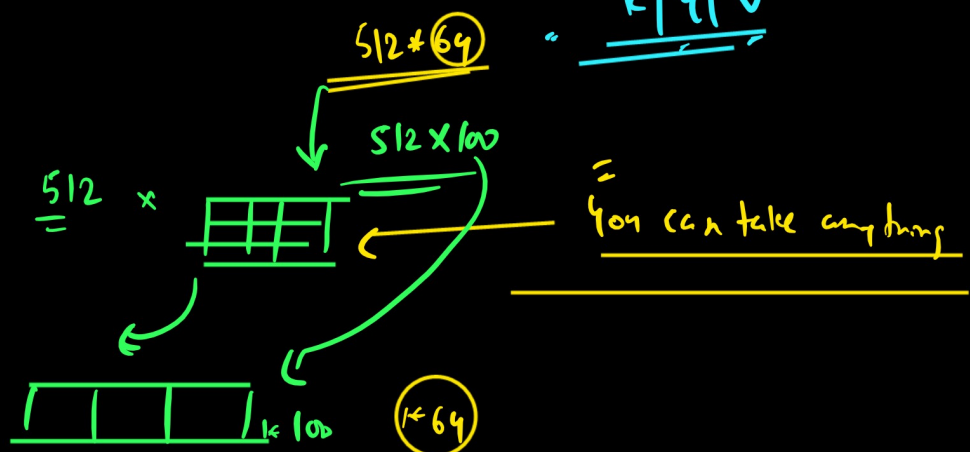
$\left( \frac{\text{normalized}}{\text{value}} \right)$

k | q | v

modified Self attention mechanism

cross / track

tomorrow



$$q * w_q$$

$$k * w_k$$

$$v * w_v$$

= Final summary final doc

Complex

$$= (\text{Lstm} \rightarrow \text{Self-Attention})$$

machine translation

$$= (\text{Encoder}) \leftrightarrow (\text{decoder})$$

Complete

Addition - (Add)  
Concatenation - (appending)

$$= \text{feed forward network} \rightarrow \text{NN} = \text{DL}$$

= Fundamental of AS model

30 min  $\rightarrow$  multihed self Attention

$$\text{normalization} \rightarrow \left[ \begin{array}{c} \downarrow \downarrow \downarrow \downarrow \downarrow \downarrow \downarrow \\ \sim \sim \sim \sim \sim \sim \sim \end{array} \right] \text{ (layer normalization)} \approx \text{learning parameter}$$

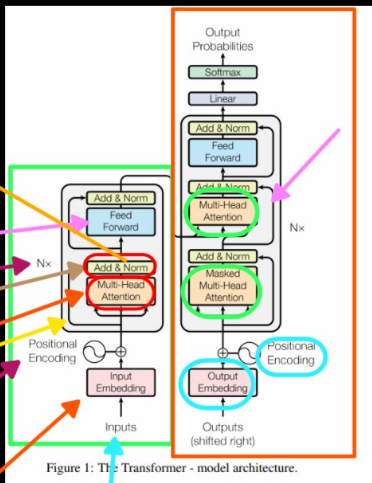
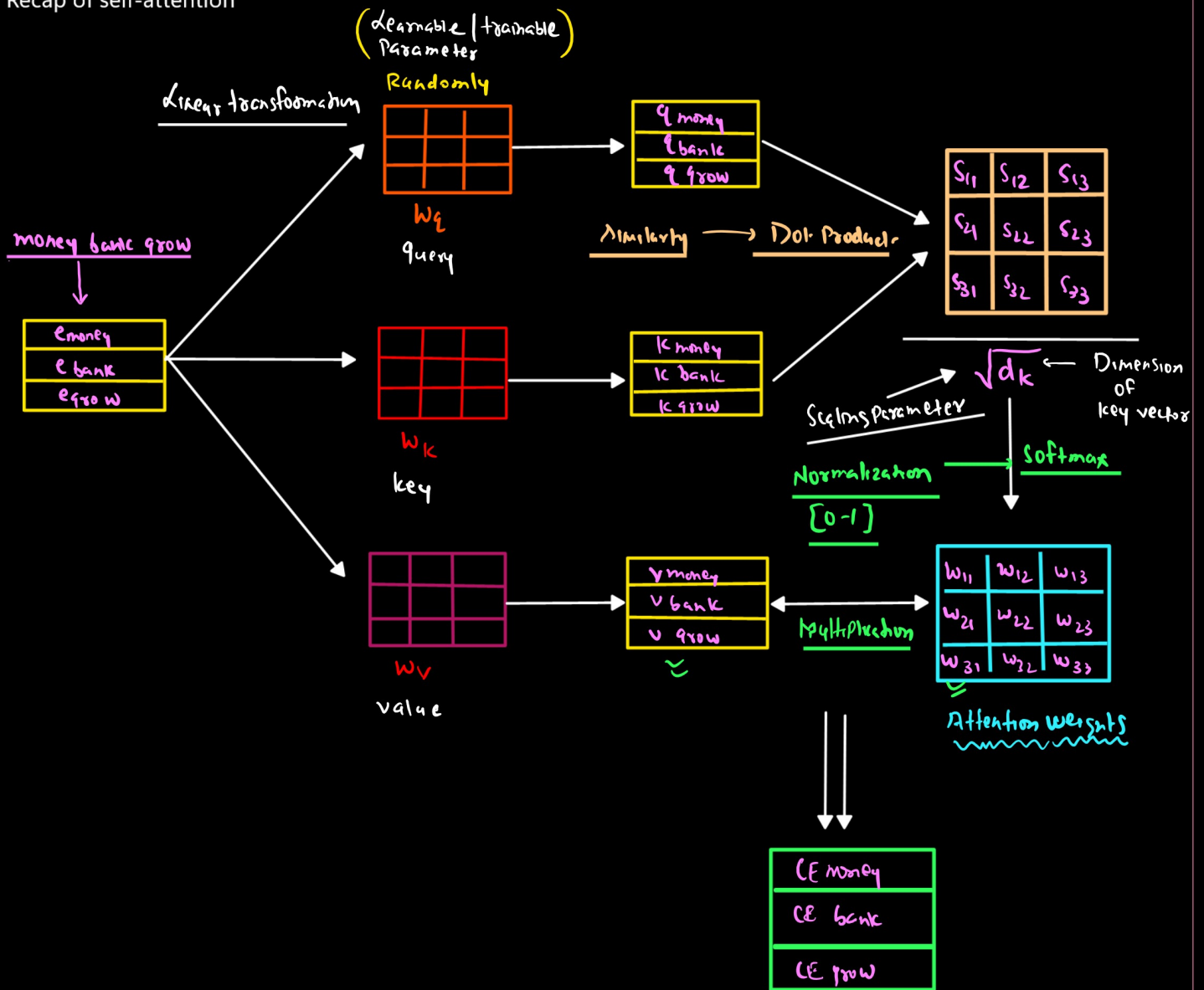


Figure 1: The Transformer - model architecture.

Transformer training is parallel, but inference is sequential due to autoregressive generation.

$$\begin{array}{c} \text{Parameter} \\ \text{(Weight)} \times \text{Attention} \\ \text{NN} \end{array}$$

# Recap of self-attention



$\sqrt{d_k}$  → Dimension of the key vector, used to scale attention scores for stability.

What it achieves

Prevents extremely large values

Keeps softmax smooth

Better gradient flow

Stable training

Softmax → Converts that scaled attention scores into probabilities (weights) that sum to 1.

$$\frac{[8, 24, 36]}{\sqrt{d_k}} = \left( \begin{bmatrix} \cdot & \cdot & \cdot \end{bmatrix} \right) \text{Softmax}$$

$$\text{[0-1]}$$



Query (Q) → What I am looking for ✓

Key (K) → What I contain / what I offer ✓

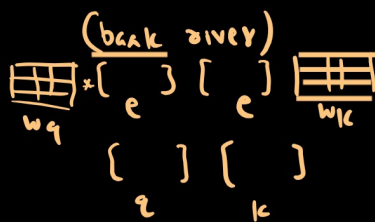
Value (V) → Actual information I provide ✓

Query vs Key → Find relevance

Score = Q.K

This computes:

How relevant is this word to me



Softmax → Convert to weights

weights = softmax(QK/sqrt(dk))

Gives importance:

high score → high attention

low score → low attention

Multiply with Value → Get final meaning

Output = weights · V

This gives: contextual representation

## Multiheaded Self Attention

A head is an self-attention mechanism defined by its  $W_Q$ ,  $W_K$ ,  $W_V$  projection matrices.

One head means single set of  $W_K$ ,  $W_Q$ ,  $W_V$  matrix

Multihead means multiple set of  $W_K$ ,  $W_Q$ ,  $W_V$  matrix

Single-head = one viewpoint

Multi-head = multiple viewpoints at the same time

Multi-head self-attention is used because a sentence can have multiple perspectives, and using multiple heads allows the model to capture different relationships and meanings in parallel.

Why Multi-Head Self-Attention?

1 Used for ambiguous sentences (NLP)

Many sentences have multiple meanings / interpretations

2 Limitation of single-head attention

Single-head attention can capture only one perspective

3 Reality in NLP

A sentence can have multiple perspectives simultaneously

Example Sentences (Ambiguity)

"The chicken is ready to eat"

Can mean:

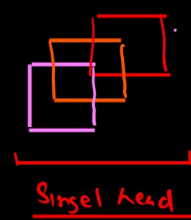
Chicken is cooked (to be eaten)

Chicken is hungry (ready to eat food)

"Man saw the astronomer with a telescope"

Ambiguity:

Who has the telescope? Man or astronomer?



Deeper context

multiple perspective

Original Research Paper

8 heads

head → hyperparameter

One head → one set of  $Q, K, V$   
multihead → multiple set of  $Q, K, V$

multiple perspective

{ Deep Context }

Why Multi-Head Helps?

Multi-head attention allows the model to:

Capture multiple relationships

Understand different meanings simultaneously

Focus on different parts of the sentence in parallel

Transformer

multiple self attention

Deeper perspective

## Paragraph

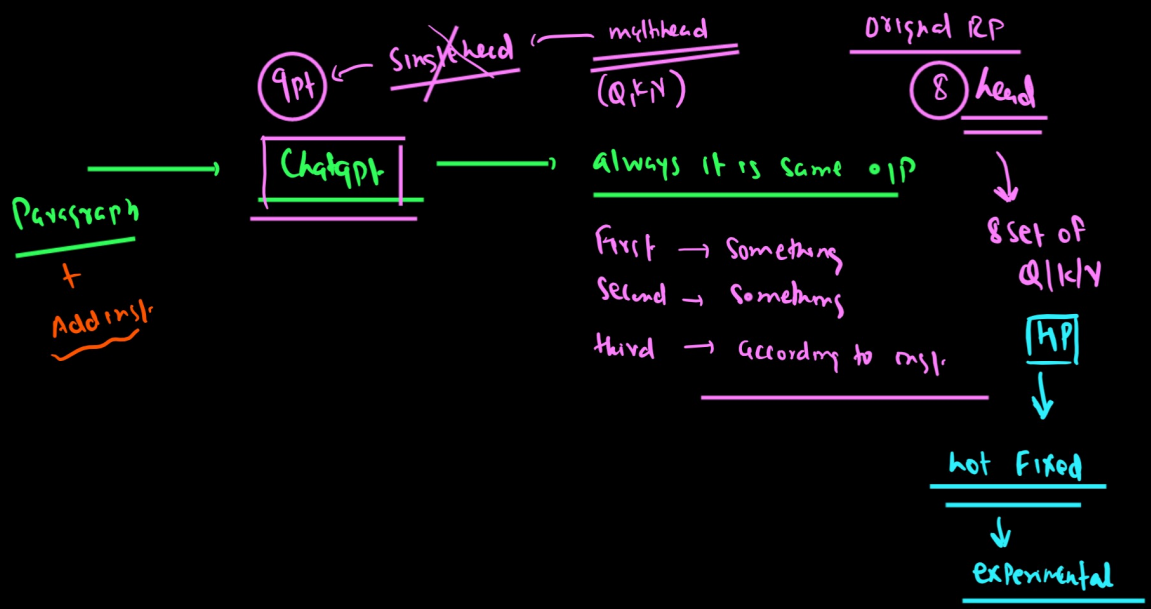
In electrical engineering, a transformer is a passive component that transfers electrical energy from one electrical circuit to another circuit, or multiple circuits. A varying current in any coil of the transformer produces a varying magnetic flux in the transformer's core, which induces a varying electromotive force (EMF) across any other coils wound around the same core. Electrical energy can be transferred between separate coils without a metallic (conductive) connection between the two circuits. Faraday's law of induction, discovered in 1831, describes the induced voltage effect in any coil due to changing magnetic flux.

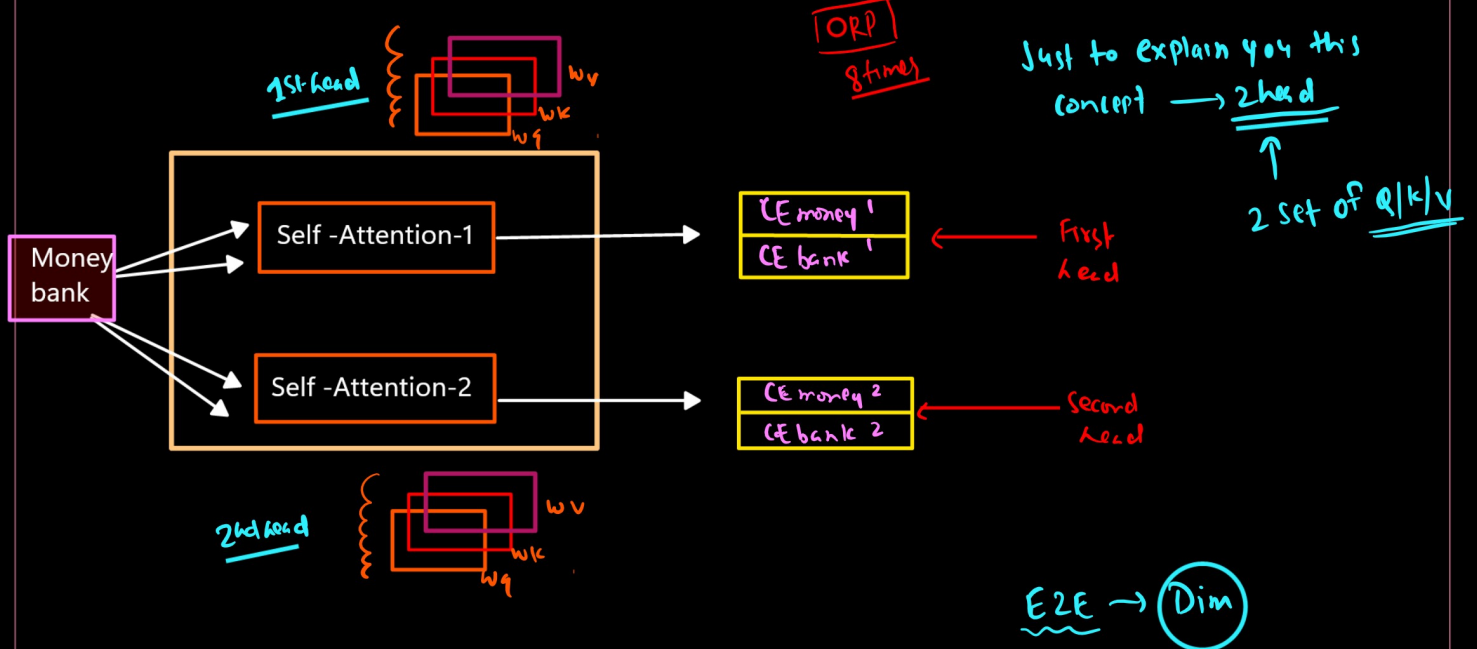
### Summary 1 one Perspective

A transformer is a passive electrical component that transfers energy between circuits via electromagnetic induction. A varying current in one coil induces a magnetic flux in the core, generating an electromotive force (EMF) in other coils. Faraday's law of induction explains this phenomenon, enabling energy transfer without direct conductive connection.

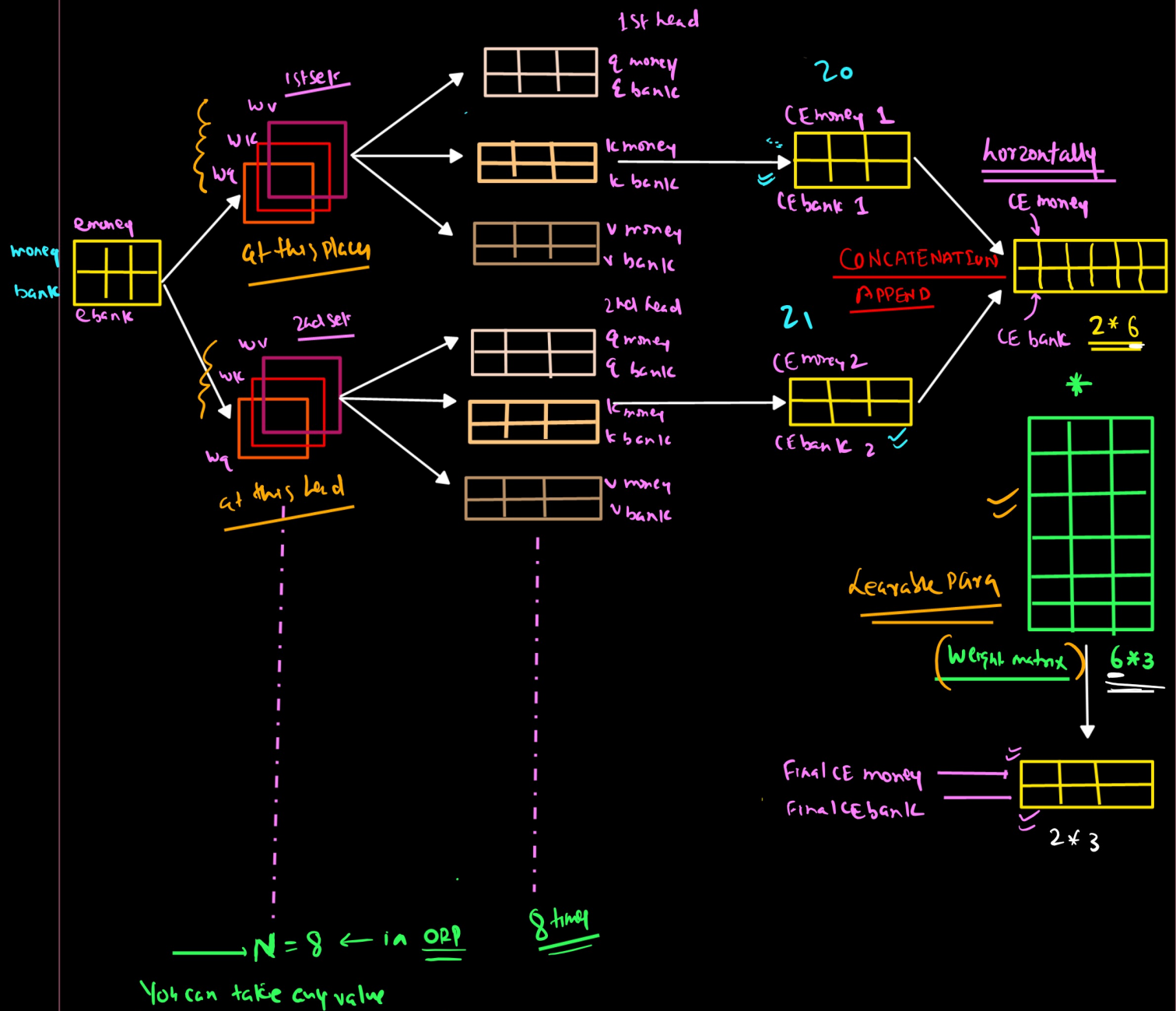
### Summary 2 another Perspective

Transformers use electromagnetic induction to transfer electrical energy between circuits. A changing current in one coil creates a magnetic flux, inducing voltage in another coil. This process, based on Faraday's law, allows energy transfer without physical connections.





One head means single set of  $W_k, W_q, W_v$  matrix  
Transformer original paper having 8 heads  
but it is hyperparameter you can have as many as head



$$\begin{matrix} \downarrow & \downarrow \\ \rightarrow & \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \end{matrix}$$

$$\begin{matrix} \rightarrow & \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \\ & \begin{matrix} R \rightarrow 2 \times 2 \leftarrow C \\ \uparrow \downarrow \downarrow \\ R \end{matrix} \end{matrix}$$

$$\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}_{3 \times 1}$$

Col matrix

vector

$$\begin{bmatrix} 1 & 2 & 3 \end{bmatrix}_{1 \times 3}$$

Row matrix

vector

Matrix multiplication

$$\begin{bmatrix} \boxed{1} & \boxed{2} \\ \boxed{3} & \boxed{4} \end{bmatrix}_{2 \times 2} * \begin{bmatrix} \boxed{3} & \boxed{4} \\ \boxed{5} & \boxed{6} \end{bmatrix}_{2 \times 2} = \begin{bmatrix} 1 \times 3 + 2 \times 5 & 1 \times 4 + 2 \times 6 \\ 3 \times 3 + 4 \times 5 & 3 \times 4 + 4 \times 6 \end{bmatrix}$$

$$= \begin{bmatrix} 13 & 16 \\ 29 & 36 \end{bmatrix}_{(2 \times 2)}$$

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}_{2 \times 3}$$

$$(m \times n)$$

\*

$$\begin{bmatrix} & & \\ & & \\ & & \end{bmatrix}_{3 \times 3}$$

$$(n \times p)$$

$$(m \times p)$$

$$= (\text{Tensor}) \leftarrow (\text{GPU})$$

$$\leftarrow (\text{Transformer})_{DL} \rightarrow (\text{matrix multiplication})$$

$$\text{GPU} \rightarrow (\text{billions})$$

$$T \left( \begin{bmatrix} & \end{bmatrix} \right) \leftarrow (\text{GPU})$$

## Cross-Attention

Cross-attention is a mechanism where one sequence attends to another different sequence to gather relevant information.

Cross-attention is a mechanism in Transformer architectures where the model uses queries from the output sequence and keys and values from the input sequence to focus on relevant parts of the input while generating the output.

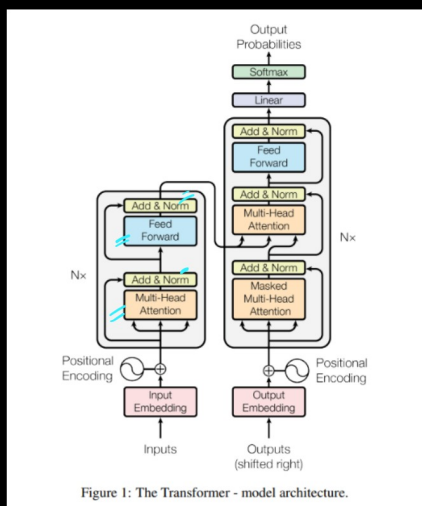
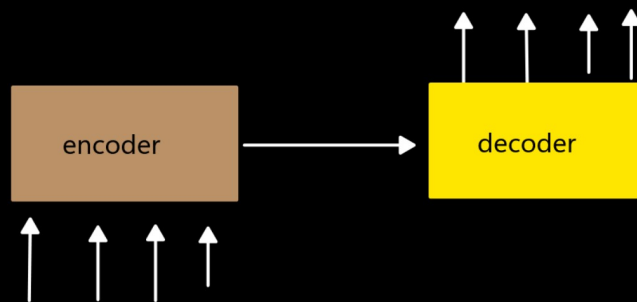


Figure 1: The Transformer - model architecture.



Self-attention looks within the same sentence, while cross-attention allows one sequence to attend to another sequence.

### Self-Attention

|       |       |
|-------|-------|
| I     | I     |
| like  | like  |
| eat-  | eat-  |
| Pizza | Pizza |

### Cross-Attention

|       |          |
|-------|----------|
| I     | मैं      |
| like  | पसंद     |
| eat-  | खाता हूँ |
| Pizza | पिज़्ज़ा |

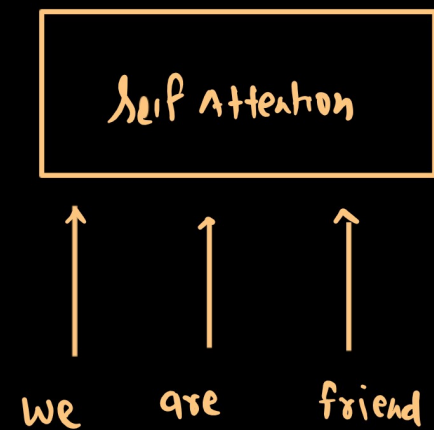
## Self-Attention vs Cross Attention

The input

The processing

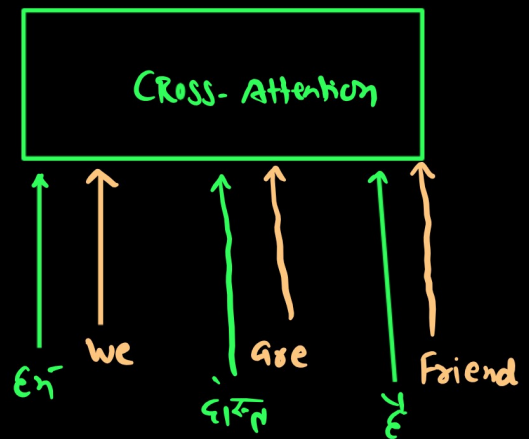
The output

### Encoder side



|        | we | are | friend |
|--------|----|-----|--------|
| we     |    |     |        |
| are    |    |     |        |
| friend |    |     |        |

### Decoder side

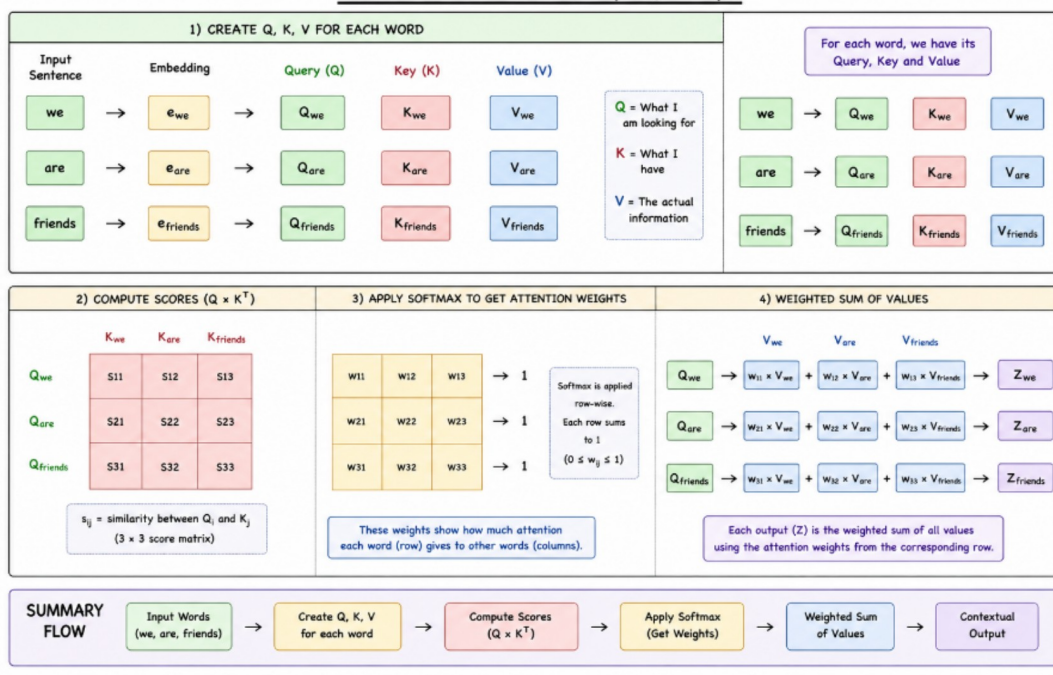


### Cross-Attention

|                  | we | are | friend |
|------------------|----|-----|--------|
| $\epsilon_n$     |    |     |        |
| $\epsilon_{1:n}$ |    |     |        |
| $\epsilon!$      |    |     |        |



## SELF-ATTENTION (Step-by-Step)

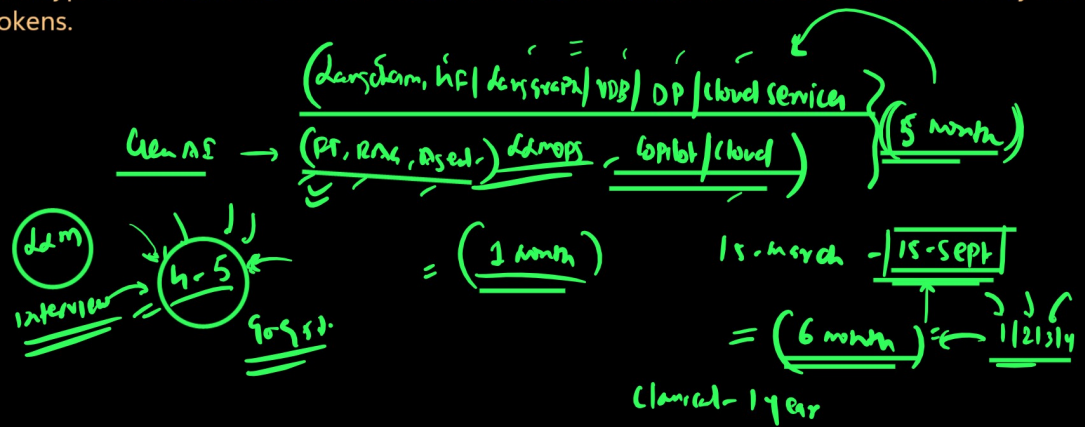


## Masked Self-Attention (Definition)

Masked self-attention is a type of self-attention where future tokens are blocked so that each token can only attend to itself and previous tokens.

## Masked Self-Attention (Definition)

Masked self-attention is a type of self-attention where future tokens are blocked so that each token can only attend to itself and previous tokens.



Layer normalization standardizes the features of each vector to improve training stability and performance.

LayerNorm normalizes a vector and scales and shifts it using learnable parameters.

### Layer Normalization Formula

$$\text{LayerNorm}(x) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \cdot \gamma + \beta$$

Std scalar  
Advanced Formula (transformer)

Meaning of each term

$x$  → input vector (e.g., token embedding)

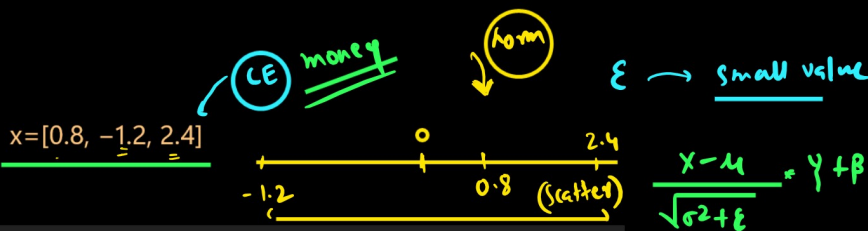
$\mu$  → mean of the vector

$\sigma^2$  → variance

$\epsilon$  → small value (avoid division by zero)

$\gamma$  → learnable scaling parameter

$\beta$  → learnable shift parameter



#### Step 1: Compute Mean

$$\mu = \frac{0.8 + (-1.2) + 2.4}{3} = \frac{2.0}{3} \approx 0.67$$

#### Step 2: Compute Variance

$$\sigma^2 = \frac{(0.8 - 0.67)^2 + (-1.2 - 0.67)^2 + (2.4 - 0.67)^2}{3}$$

$$= \frac{(0.13)^2 + (-1.87)^2 + (1.73)^2}{3} = \frac{0.0169 + 3.4969 + 2.9929}{3} = \frac{6.5067}{3} \approx 2.17$$

#### Step 3: Normalize

$$x_{\text{norm}} = \frac{x - \mu}{\sqrt{\sigma^2}}$$

$\sqrt{2.17} \approx 1.47$

Now:

- $(0.8 - 0.67) / 1.47 \approx 0.09$
- $(-1.2 - 0.67) / 1.47 \approx -1.27$
- $(2.4 - 0.67) / 1.47 \approx 1.18$

$[0.09, -1.27, 1.18]$

normalize

compress

#### Step 4: Scale & Shift

$$y = x_{\text{norm}} \cdot \gamma + \beta$$

$\gamma = [1, 1, 1], \beta = [0, 0, 0]$

(during learning) optimizer

$[0.09, -1.27, 1.18] = \text{training}$

$\gamma$  → scale parameter

$\beta$  → shift parameter

$$\begin{aligned} \gamma &\leftarrow \gamma - \text{learning\_rate} * \text{gradient} \\ \beta &\leftarrow \beta - \text{learning\_rate} * \text{gradient} \end{aligned}$$

optimizer  
BP

{gradient  
descent}

DL

DL

(optimizer)  
AdamW

### 1. Stabilizes Training

Keeps values in a controlled range → prevents exploding or very large activations.

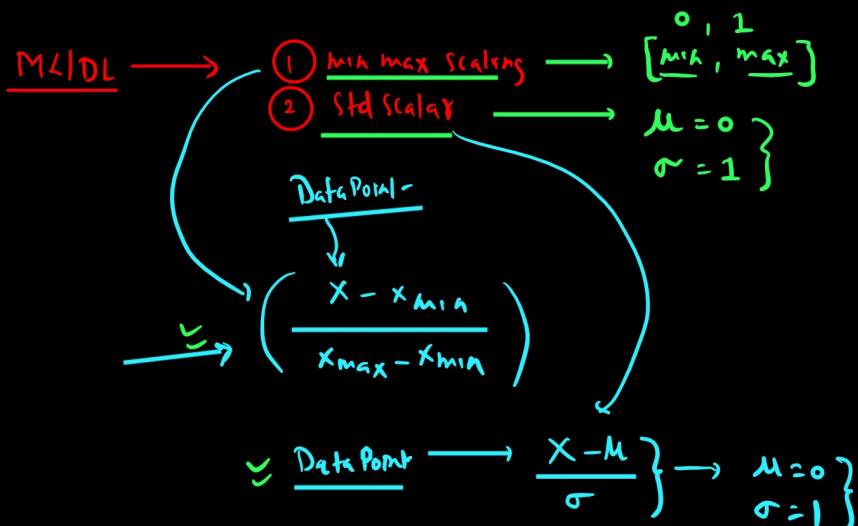
### 2. Improves Gradient Flow

Avoids vanishing/exploding gradients → helps deep transformers train smoothly.

### 3. Faster Convergence

Model learns faster because inputs to each layer are well-conditioned (consistent scale).

Normalization → Compensate the value within the specific range



Layer Normalization

↓

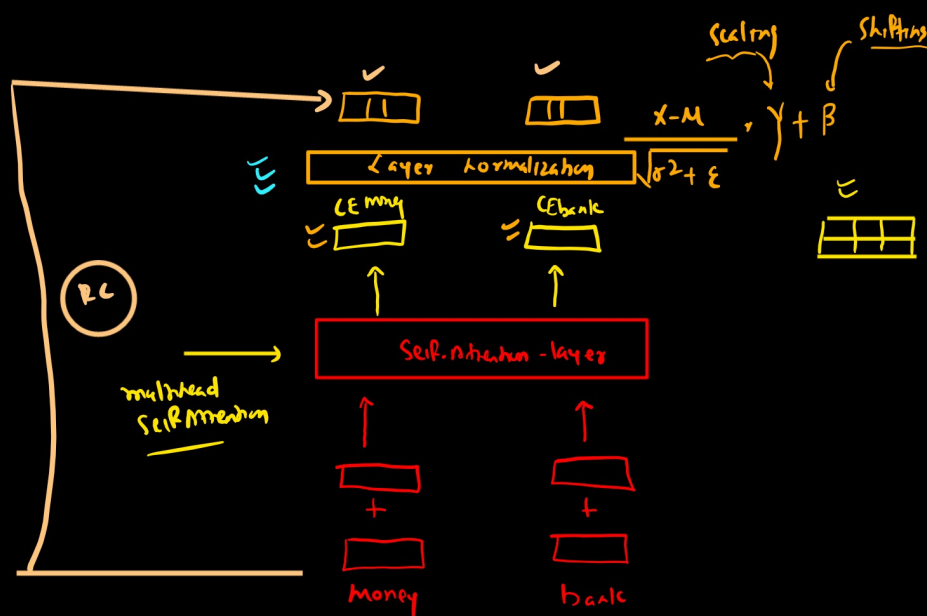
$\left\{ \begin{matrix} \text{all the features} \\ \text{in a vector} \end{matrix} \right\}$      range (compensate)

$\left[ \begin{matrix} 1 & 1 & 1 & 1 & 1 & 1 \\ 5 & 6 & 8 & 9 & 1 & 2 \end{matrix} \right]$

↑

1 × 6

(layer normalization)



$\beta$  gives the model the freedom to choose where the data should be centered.

Shifting Param

$$\left\{ \frac{\text{norm-stdev-stakey}}{\text{mean}} \right\}$$

$\sigma = 1$

All the concept of encoder

